

Keycloak

Stefan Reuter

Version 2.1-10-g50bb10c, 30.03.2026

TRION

Inhaltsverzeichnis

1. Einführung	1
Was ist Keycloak?	1
Keycloak ist Open Source	1
Features	1
Informationsquellen	2
Releases und Community	2
Releaseplanung	2
2. Installation	4
Development Mode	4
Exkurs: Docker Compose	4
Übung 01: Keycloak starten	4
Admin User & Admin Console	4
Konfigurationsoptionen	5
Eigenschaften Dev Mode	5
Production Mode	5
Beispiel Prod Mode	5
TLS-Zertifikat über ACME erzeugen	6
Reverse Proxy	6
Traefik als Reverse Proxy	7
Übung 02: Traefik	7
Tipps bei Timeout	7
Datenbank	7
PostgreSQL als Container	8
Datenbank konfigurieren	8
Übung 03: PostgreSQL	8
Optimierter Keycloak Build	9
kc.sh build	9
Optimiertes Container Image bauen	9
Keycloak Cluster	9
Caches	9
Konfiguration der Caches	10
Kubernetes Transport Stack	10
Installation in Kubernetes	10
Multi-Site Deployments	10
3. Realms	12
Was ist ein Realm?	12
Welche Realms gibt es?	12
Realm anlegen	12

Realm konfigurieren	12
Übung 04: Realm anlegen	13
Authentication konfiguration	13
Password Policy	13
Antipattern: Erzwungene Passwortänderung	14
Übung 05: Selbstregistrierung	15
4. Benutzer, Rollen & Gruppen	16
Benutzer anlegen	16
Rollen	16
Zuweisen von Rollen	16
Übung 06: Benutzer mit Rollen	16
Gruppen	16
Typische Nutzung	17
LDAP & AD	17
Synchronisation	17
Übung 07: Anbindung AD	17
Mapping von LDAP-Attributen	18
LDAP Filter	18
Gruppen & Rollen	18
Externe Identityprovider	18
Beispiel: GitHub	18
Register Application	19
Client Id & Secret	19
Provider hinzufügen	19
Sign in with GitHub	20
Verknüpfung der Accounts	20
Übung 08: Social Login	21
Organisationen	21
Kernfunktionen von Keycloak Organizations	21
Organisationen aktivieren und anlegen	22
Mitglieder verwalten	22
Identity Provider zuweisen	22
Roadmap Organisationen	22
5. Clients	23
Client anlegen	23
Übung 09: Public Client	23
Konfiguration auf Client-Seite	23
Logout	23
Client Registration Service	24
6. Mehrfaktorthauthentifizierung	25
MFA	25

TOTP/HOTP	25
Übung 10: TOTP	25
WebAuthn	25
Authentication Flows	25
Übung 11: WebAuthn	26
Individuelle Lösungen	26
7. Themes	27
Organisation von Themes	27
Theme Types	27
Woraus besteht ein Theme?	27
Entwicklung von Themes	27
theme.properties	28
Beispiel: Custom CSS für Login	28
Übung 12: Theme	28
Tipps & Tools	29
8. Plugins	30
Erweiterung von Keycloak	30
Tipps zur Entwicklung	30
Service Provider Interfaces	30
Provider Info	30
Authentication SPI	31
Action Token Handler SPI	31
Event Listener SPI	31
SAML Role Mappings SPI	32
Vault SPI	32
Erweiterungen des Servers	32
9. REST API für Administration	33
Admin CLI	33
REST API	33
Export und Import	33
10. Anpassungen für Produktivbetrieb	35
Tipps	35
Metriken und Monitoring	35
Feature Flags	35
Kubernetes	35

Kapitel 1. Einführung

Was ist Keycloak?

- Single-Sign-On-Lösung für Webanwendungen und RESTful Web-Services
- Zentrale und eigenständige Komponente in der Anwendungslandschaft für Authentifizierung
- Anpassbares UI für Login, Registrierung, Administration und Account-Verwaltung
- Kann an LDAP oder Active Directory angebunden werden
- Unterstützt Delegation der Authentifizierung z.B. an Facebook oder Google ("Login mit Facebook")

Keycloak ist Open Source

- Keycloak ist Open-Source-Software unter der Apache Lizenz
- [Incubation Projekt](#) bei der Cloud Native Computing Foundation
- Support vom Open-Source-Projekt gibt es jeweils **nur** für die letzte veröffentlichte Version
- Kommerzieller Support erhältlich z.B. als [Red Hat build of Keycloak](#) und von unabhängigen Anbietern



open source
initiative®

Features

- Single-Sign On und Single-Sign Out für Browser-Anwendungen (Authentifizierung)
- Support für OpenID Connect und OAuth 2.0
- Support für SAML
- Identity Brokering: Unterstützung externer OpenID Connect oder SAML IdPs

- Social Login (Google, GitHub, Facebook, etc.)
- User Federation: Synchronisation der Benutzer mit LDAP oder AD
- Kerberos Bridge
- Admin Console
- Account Management Console für Benutzer
- Anpassbare Themes
- 2FA: TOTP/HOTP über Google Authenticator oder FreeOTP
- Unterschiedliche Login-Flows: Registrierung, Passwort vergessen, E-Mail-Adresse bestätigen, erzwungenes Passwort-Update, ...
- Session Management: Admins und Benutzer können aktive Sessions sehen und verwalten
- Token Mapper, um Attribute von Benutzern, Rollen, etc. in Tokens zu mappen

Informationsquellen

- Keycloak Homepage: <https://www.keycloak.org/>
- Offizielle [Dokumentation](#)
 - [Server Administration](#)
 - [Server Developer](#) für Themes und Plugins
 - [Upgrading Guide](#)
 - [Admin REST API](#)
- [Guides](#)
- Ankündigungen im [Blog](#)

Releases und Community

- [GitHub Organisation](#) mit Quellcode
- [Releases](#), [Nightly Build](#) und geplante [Milestones](#)
- Container Images bei [Quay.io](#)
 - [latest](#), [nightly](#) und Tags mit spezifischer Versionsnummer
- [Release Notes](#)
- Mailing-Listen [keycloak-user](#) und [keycloak-dev](#)
- GitHub [Diskussionsforen](#)

Releaseplanung

- **Vor Version 26**
 - Ca. **4 Major-Releases** pro Jahr
 - Steigende Anzahl von breaking Changes

- Hoher Aufwand Keycloak-Installationen up-to-date zu halten

- **Seit Version 26**

- Geplant sind **4 Minor-Releases** pro Jahr
- Ein Major-Release alle 2 bis 3 Jahre
- Client-Bibliotheken sollen separat versioniert werden
- Breaking Changes bei Minor-Releases sollen Opt-In sein

Kapitel 2. Installation

Development Mode

- Einfachste Möglichkeit Keycloak zu starten, Command: `start-dev`
- Username und Passwort für Administrator über Umgebungsvariablen setzen
- HTTP Port 8080 mappen und von außen verfügbar machen
- `docker-compose.yml`:

```
---
services:
  keycloak:
    image: quay.io/keycloak/keycloak:26.5.6
    # image: registry.redhat.io/rhbk/keycloak-rhel9:26.4
    command: start-dev
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
    ports:
      - "8080:8080"
```

Exkurs: Docker Compose

- Starten über `docker compose up` bzw. `docker compose up -d`
 - Kommandozeilenoption `-d` startet den Container im Hintergrund
- Logs ansehen: `docker compose logs`
- Restart: `docker compose restart`
- Beenden und Löschen: `docker compose down`

Übung 01: Keycloak starten

- Erstelle ein neues Verzeichnis `uebung01` und lege darin eine Datei `docker-compose.yml` an, mit der du Keycloak im Development Mode starten kannst.
- Starte Keycloak und rufe die Oberfläche im Webbrowser auf.
- Melde dich mit dem konfigurierten Admin-Account an der Admin-Console an.

Admin User & Admin Console

- Admin-User und Admin-Passwort können interaktiv über Browser festgelegt werden, wenn Keycloak direkt über "localhost" erreichbar ist (nicht mit Docker).
- Alternativ: über Umgebungsvariablen definieren (`KEYCLOAK_ADMIN` und `KEYCLOAK_ADMIN_PASSWORD`).

- Aufruf der Admin Console über Browser unter <http://localhost:8080/admin>

Konfigurationsoptionen

- Konfigurationsoptionen können über folgende Mechanismen gesetzt werden:
 - Kommandozeilenparameter
 - Umgebungsvariablen
 - Konfigurationsdateien
- Wird Keycloak als Container betrieben, bietet sich die Nutzung von Umgebungsvariablen an
- Details finden sich im Server Guide [Configuring Keycloak](#)
- Eine Liste aller Konfigurationsoptionen stellt das Keycloak-Projekt unter <https://www.keycloak.org/server/all-config> bereit

Eigenschaften Dev Mode

- Der Development Mode kommt mit folgenden Standardeinstellungen:
 - HTTP auf Port 8080 ist aktiviert (kein TLS)
 - "Strict hostname resolution" ist deaktiviert
 - Caching ist auf "local" gesetzt, d.h. es wird kein verteilter Cache für Hochverfügbarkeit genutzt
 - Caching von Themes und Templates ist deaktiviert

Production Mode

- Starten mit Command `start` start `start-dev`
- Benötigt TLS
 - Keycloak benötigt TLS-Zertifikat und privaten Schlüssel, falls es TLS selbst terminieren soll
 - Alternativ: Verwendung eines vorgeschalteten Reverse Proxies zum Terminieren von TLS
- Hostname bzw. URL muss über `KC_HOSTNAME` gesetzt werden

Beispiel Prod Mode

- `KC_HOSTNAME` setzt den URL, unter dem Keycloak angesprochen wird (strict hostname resolution)
- `KC_HTTPS_CERTIFICATE_FILE` und `KC_HTTPS_CERTIFICATE_KEY_FILE` geben an, in welchen Dateien sich TLS-Zertifikat und privater Schlüssel befinden

```
services:
  keycloak:
    image: quay.io/keycloak/keycloak:26.5.6
    command: start
```

```
environment:
  KC_HOSTNAME: https://keycloak.local.2qn.org:8443/
  KC_HTTPS_CERTIFICATE_FILE: /certs/cert.pem
  KC_HTTPS_CERTIFICATE_KEY_FILE: /certs/privkey.pem
  KEYCLOAK_ADMIN: admin
  KEYCLOAK_ADMIN_PASSWORD: admin
ports:
  - "8443:8443"
volumes:
  - ./certs:/certs
```

TLS-Zertifikat über ACME erzeugen

- TLS-Zertifikate für Keycloak lassen sich von einer öffentlichen CA beziehen, die das ACME-Protokoll unterstützt (z.B. Let's Encrypt oder ZeroSSL)
- Das Kommandozeilentool `acme.sh` kann solche Zertifikate einfach erstellen:

```
docker run --rm -it -v "$(pwd)/acme.sh:/acme.sh" -p 80:80 -p 443:443 neilpang/acme.sh \
  --register-account -m zeross1@example.com

docker run --rm -it -v "$(pwd)/acme.sh:/acme.sh" -p 80:80 -p 443:443 neilpang/acme.sh \
  --issue -d keycloak.example.com --standalone
```

- Ergebnis:

```
keycloak.example.com_ecc/ca.cer
keycloak.example.com_ecc/fullchain.cer          # <-- Certificate Chain
(cert.pem)
keycloak.example.com_ecc/keycloak.example.com.cer
keycloak.example.com_ecc/keycloak.example.com.conf
keycloak.example.com_ecc/keycloak.example.com.csr
keycloak.example.com_ecc/keycloak.example.com.csr.conf
keycloak.example.com_ecc/keycloak.example.com.key # <-- Private Key (privkey.pem)
```

Reverse Proxy

- Zwei Varianten bei Nutzung von Reverse Proxy:
 - **forwarded**: Der **Forwarded** Header wird gemäß RFC7239 ausgewertet
 - **xforwarded**: Die nicht standardisierten (aber weit verbreiteten) **X-Forwarded-*** Header werden verwendet (z.B. **X-Forwarded-For**, **X-Forwarded-Proto**, **X-Forwarded-Host** und **X-Forwarded-Port**)
- Welche Header verwendet werden, wird über Konfigurationsoption **proxy-headers** festgelegt

- Soll die Kommunikation zwischen Reverse Proxy und Keycloak unverschlüsselt über HTTP erfolgen, muss zusätzlich die Option `http-enabled` auf `true` gesetzt werden
- Details finden sich im Server Guide [Using a reverse proxy](#)

Traefik als Reverse Proxy

- Moderner Reverse Proxy, besonders gut geeignet in Umgebungen mit Docker und Kubernetes
- Konfiguration erfolgt über Kommandozeilenparameter, Konfigurationsdateien und Labels an Docker-Containern
- Unterstützt automatisches Redirect von HTTP auf HTTPS
- Kann TLS Zertifikate über ACME-API (RFC 8555) z.B. über Let's Encrypt erzeugen oder ein extern erzeugtes Zertifikat nutzen.
- Fallstricke gibt es je nach eingesetzten Versionen bei der Nutzung von mTLS ([traefik#9669](#), [keycloak#17171](#), [keycloak#46395](#))

Übung 02: Traefik

- Beende den Docker-Container aus Übung 01.
- Setze Keycloak mit vorgeschaltetem Reverse Proxy im Prod Mode auf. Nutze dafür die zur Verfügung gestellte `docker-compose.yml`-Datei und die Traefik-Konfigurationsdatei `tls.yml` im Verzeichnis `uebung02`.
- Erzeuge ein TLS-Zertifikat für den Hostnamen deiner Schulungs-VM.
- Kopiere den Private Key nach `certs/privkey.pem` und die Zertifikatskette nach `certs/cert.pem`.
- Starte die Docker-Container und rufe die Admin-Konsole auf. Prüfe, dass TLS mit dem korrekten Zertifikat verwendet wird.

Tipps bei Timeout

- Tritt beim Laden der Admin-Console ein Timeout auf, ist häufig der Hostname bzw. URL (Umgebungsvariable `KC_HOSTNAME`) nicht richtig gesetzt.
- Prüfe insbesondere das Schema `http` vs. `https` und ggf. den Port.
- Beende Keycloak mit `docker-compose down` und führe eine erneute Initialkonfiguration über `docker compose up` durch.

Datenbank

- In der Standardkonfiguration startet Keycloak mit einer H2-Datenbank (`dev-file`), die Daten werden innerhalb des Containers gehalten und gehen verloren, sobald der Container gelöscht wird
- Für den Prod unterstützt Keycloak folgende Datenbanken:
 - MariaDB Server (`mariadb`) und MySQL (`mysql`)

- Microsoft SQL Server (`mssql`)
- Oracle Database (`oracle`)
- PostgreSQL (`postgres`) bzw. CockroachDB
- Der Typ der Datenbank wird über den Parameter `db` bzw. die Umgebungsvariable `KC_DB` gesetzt.
- Details finden sich im Server Guide [Configuring the database](#)

PostgreSQL als Container

- `docker-compose.yml`:

```
---
services:
# ...
  db:
    image: postgres:18.2
    restart: unless-stopped
    environment:
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: supersecretpassword
    volumes:
      - ./db:/var/lib/postgresql
```

- Zugriff auf Datenbankkonsole über
`docker compose exec db psql -U keycloak`

Datenbank konfigurieren

- `KC_DB`: Typ der Datenbank, z.B. `postgres`
- `KC_DB_URL`: JDBC-URL, z.B. `jdbc:postgresql://db/keycloak`
- `KC_DB_USERNAME`: Benutzername für den Datenbankzugang
- `KC_DB_PASSWORD`: Passwort für den Datenbankzugang

Übung 03: PostgreSQL

- Beende den Docker-Container aus Übung 02.
- Setze Keycloak mit PostgreSQL als Datenbank auf. Nutze dafür die zur Verfügung gestellte `docker-compose.yml`-Datei im Verzeichnis `uebung03`.
- Über den Private Key und das TLS-Zertifikat aus der vorherigen Übung (Kopieren von `uebung02/certs` nach `uebung03/certs`).
- Starte die Docker-Container und prüfe die Admin-Konsole.
- Verbinde dich über die Datenbankkonsole mit der Datenbank und prüfe, dass die Tabellen von Keycloak (z.B. `user_entity`) dort angelegt wurden

Optimierter Keycloak Build

- Beim Starten von Keycloak über `start-dev` oder `start` wird jeweils zusätzlich ein `build` ausgeführt.
- Beim Build werden einige Optimierungen für den Start und die Laufzeit vorgenommen ([Quarkus re-augmentation](#)).
- Der Build kann einige Sekunden dauern und verzögert damit den Start, was insbesondere in dynamischen Umgebungen wie Kubernetes unerwünscht ist.
- Lösung: Build explizit vorab durchführen (z.B. in CI-Pipeline) und optimiertes Container-Image publizieren.

kc.sh build

- Bauen mit `kc.sh build <build optionen>`
 - <https://www.keycloak.org/server/all-config?f=build>
- Starten mit `kc.sh start --optimized <config optionen>`
 - <https://www.keycloak.org/server/all-config?f=config>

Optimiertes Container Image bauen

- Dockerfile:

```
FROM quay.io/keycloak/keycloak:26.5.6

RUN /opt/keycloak/bin/kc.sh build \
    --db=postgres \
    --health-enabled=true \
    --metrics-enabled=true
```

- Siehe auch [Running Keycloak in a container](#)

Keycloak Cluster

- Keycloak kann im Cluster betrieben werden
- Höhere Verfügbarkeit
- Höherer Durchsatz

Caches

- Keycloak nutzt neben der Datenbank als (verteilter) Cache [Infinispan](#) (high-performance, distributable in-memory data grid)
 - Lokale Caches für Realms, User, Authorization und externe öffentliche Schlüssel

- Replicated Cache (auf alle Nodes repliziert) für Invalidierungsnachrichten
- Distributed Caches für Sessions (auth, user, clients, usw.), fehlgeschlagene Logins, Action Tokens
- Default Konfiguration für Infinispan in `/opt/keycloak/conf/cache-ispn.xml`

Konfiguration der Caches

- Verschiedene Optionen
 - Eigene Infinispan Konfigurationsdatei über `KC_CACHE_CONFIG_FILE` setzen
 - Externen Infispan-Server verwenden (`KC_CACHE_REMOTE_HOST`, `_PORT`, `_USERNAME` und `_PASSWORD`)
 - Default Konfiguration verwenden und Transport Stack festlegen (`KC_CACHE_STACK`): `tcp`, `udp`, `kubernetes` und Cloud-Provider spezifische Stacks für AWS, Google und Azure
- Guide: [Configuring distributed caches](#)



Kommunikation der Caches untereinander sollte abgesichert werden, wenn das Netzwerk nicht sicher ist. Siehe [Securing cache communication](#).

Kubernetes Transport Stack

- Nutzt zur Discovery `DNS_PING`, d.h. ermittelt Cluster-Nodes aus `A`- oder `SRV`-Records
- Nutzt TCP als Transportprotokoll
- Gut geeignet für Betrieb in Kubernetes, Docker Swarm aber auch in anderen Umgebungen, wenn Zugriff auf DNS besteht
- `KC_CACHE_STACK: kubernetes`
- Zusätzlich muss der abzufragende DNS-Record über `-Djgroups.dns.query=` gesetzt werden

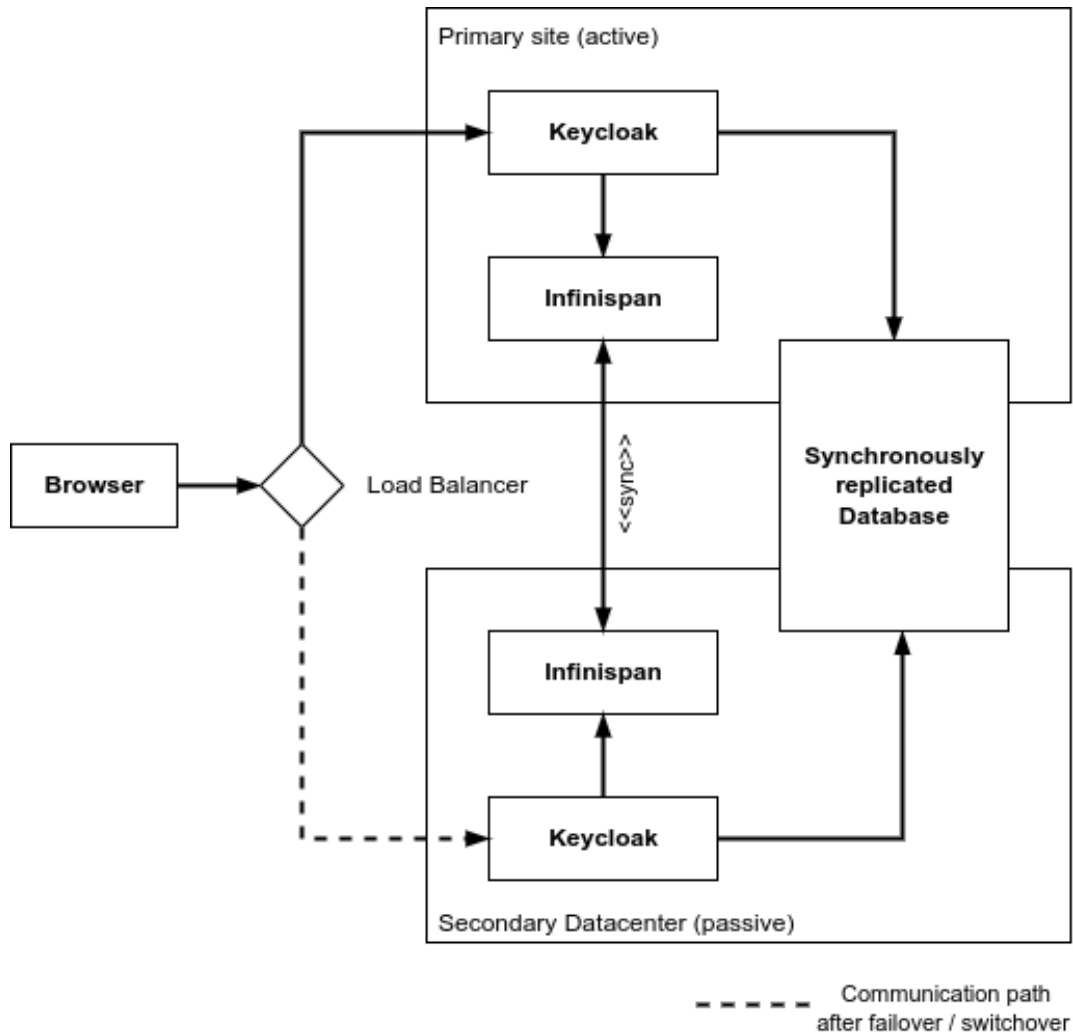
Installation in Kubernetes

- [Kubernetes Operator für Keycloak](#)
- Einfaches Verwalten von Keycloak Clustern
- Bereitgestellte CRDs:
 - `Keycloak` für Keycloak Cluster
 - `KeycloakRealmImport`
- Admin Passwort: `kubectl get secret example-kc-initial-admin -o json-path='{.data.password}' | base64 --decode`

Multi-Site Deployments

- Synchrone Replikation in zweites Rechenzentrum

- In jedem RZ können mehrere Keycloak-Nodes laufen
- Externer Infinispan Cache
- Synchron replizierte Datenbank
- Vorschalteter Loadbalancer schwenkt bei Ausfall des primären RZ
- Guide: [Multi-site deployments](#)



Kapitel 3. Realms

Was ist ein Realm?

- Ein Realm verwaltet eine Menge von Benutzern, Credentials, Rollen, Gruppen und Clients.
- Ein Benutzer gehört zu einem Realm und meldet sich an einem Realm an.
- Realms sind **voneinander isoliert** und können nur auf die Benutzer zugreifen, die sie verwalten.

Welche Realms gibt es?

Master-Realm

Wurde bei der Installation angelegt und beinhaltet den Admin-Benutzer, sollte nur verwendet werden, um andere Realms zu verwalten.

Andere Realms

Werden vom Admin im Master-Realm angelegt und beinhalten die Benutzer, Clients, etc.

Realm anlegen

- Als Admin über die Admin Console im Master-Realm oben links im Drop Down "Create Realm" auswählen und Namen festlegen
- Bei der Planung überlegen, welche Aufteilung sinnvoll ist, da Benutzer und Anwendungen (Clients) jeweils voneinander isoliert sind
- Beispiele:
 - Realm für interne Benutzer (Mitarbeiter), die auf interne Anwendungen zugreifen und Realm für Kunden, die auf Anwendungen für Kunden zugreifen
 - Unterschiedliche Realms für B2B- und B2C-Kunden, wenn diese jeweils unterschiedliche Anwendungen verwenden
 - Realms für unterschiedliche Umgebungen (Dev, Test, Prod)

Realm konfigurieren

- Nach Anlegen des Realms unter "Configure/Realm settings"
- Require SSL: Erzwingt SSL für gar keine, alle oder externe (nicht localhost oder RFC-1918-Adressbereiche) Requests
- Links zu
 - OpenID Endpoint Configuration
 - SAML 2.0 Identity Provider Metadata
- Login-Einstellungen (z.B. Self registration, Passwort vergessen, Remember me)
- Anbindung SMTP-Server für E-Mail-Versand

- Auswahl Themes und Lokalisierung (Festlegen verfügbarer Sprachen, Standardsprache, Texte anpassen)
- Keys
- Event Listener
 - [E-Mail](#): informiert Nutzer über Ereignisse wie Lock-Out oder Passwortänderung
- Events können für Auditzwecke gespeichert werden, optionale Expiration für automatisches Löschen
 - User event settings
 - Admin event settings
- Gültigkeitszeiträume und Timeouts für Sessions und Tokens
- Client Policies
 - Profiles (insbesondere Financial-grade API Security Profile)
 - Policies mit konfigurierten Conditions
- Default Rollen und Gruppen für Benutzer, die über die Registrierung neu angelegt werden

Übung 04: Realm anlegen

- Lege einen neuen Realm **training** an
- Aktiviere die Funktionen "Passwort vergessen" und "Remember me"
- Konfiguriere die E-Mail-Einstellungen für den neuen Realm (Host: **mailcatcher**, Port: **1025**)
- Aktiviere das Speichern von User- und Admin-Events zu Auditzwecken mit einer Haltedauer von jeweils 30 Tagen

Authentication konfiguration

- Authentication
 - Flows (Built-in und custom)
 - Required Actions
 - Policies (Password, WebAuthn, OTP)

Password Policy

Passwort *



Ihr Passwort muss folgende Voraussetzungen erfüllen:

- ✗ Muss mindestens 14 Zeichen enthalten. (nicht erfüllt)
- ✗ Muss mindestens eine Ziffer '0-9' enthalten. (nicht erfüllt)
- ✗ Muss mindestens einen Großbuchstaben enthalten. (nicht erfüllt)
- ✗ Muss mindestens einen Kleinbuchstaben enthalten. (nicht erfüllt)
- ✗ Muss mindestens ein Sonderzeichen aus '/', ':', ';', '#', '+', '@', '&', '!', '?', '-' enthalten. (nicht erfüllt)
-  ✗ Darf kein anderes Sonderzeichen als '/', ':', ';', '#', '+', '@', '&', '!', '?', '-' enthalten. (nicht erfüllt)
- ✗ Die ersten drei Zeichen dürfen nicht mit den ersten drei Zeichen Ihres Benutzernamens übereinstimmen. (nicht erfüllt)
- ✗ Unter den ersten drei Zeichen dürfen keine Dopplungen vorkommen (bspw. XX, xx, XaX). (nicht erfüllt)
- ✗ Die ersten drei Zeichen dürfen kein Leerzeichen sein. (nicht erfüllt)
- ✗ Das erste Zeichen darf weder '?', '!' noch ein Leerzeichen sein. (nicht erfüllt)

- Quelle: [Zentrale Rechnungseingangsplattform des Bundes \(ZRE\)](#)
- Humorvolle API: <https://passwordpurgatory.com/>
- Mindestens aktivieren:
 - Not Username
 - Not Email
 - Minimum Length

Antipattern: Erzwungene Passwortänderung

- [BSI ORP.4 Identitäts- und Berechtigungsmanagement A23](#)
 - "IT-Systeme oder Anwendungen SOLLTEN NUR mit einem validen Grund zum Wechsel des Passworts auffordern. Reine zeitgesteuerte Wechsel SOLLTEN vermieden werden. Es MÜSSEN Maßnahmen ergriffen werden, um die Kompromittierung von Passwörtern zu erkennen. Ist dies nicht möglich, so SOLLTE geprüft werden, ob die Nachteile eines zeitgesteuerten Passwortwechsels in Kauf genommen werden können und Passwörter in gewissen Abständen gewechselt werden."
- [NIST SP800-63B](#)
 - "Verifiers and CSPs SHALL NOT require users to change passwords periodically. However, verifiers SHALL force a change if there is evidence of compromise of the authenticator."

Übung 05: Selbstregistrierung

- Benutzer sollen sich selbst einen Account anlegen können.
- Bei der Registrierung soll die E-Mail-Adresse verifiziert werden.
- Passwörter dürfen weder dem Benutzernamen, noch der E-Mail-Adresse entsprechen, müssen mindestens 10 Zeichen lang sein und mindestens eine Ziffer enthalten.
- Konfiguriere den Realm entsprechend und registriere dich als neuer Benutzer über die Account Console unter https://HOST_NAME/realms/training/account.
Der Sign-in Button befindet sich oben rechts.

Kapitel 4. Benutzer, Rollen & Gruppen

Benutzer anlegen

- In der Admin Console über "Users/Add user" im richtigen Realm
- Eindeutiger Username ist pflicht
- Required user actions: Aktionen, die der Benutzer beim nächsten Login durchführen muss
- Attributes: beliebige Key-Value-Paare mit eigenen Werten
- Credentials: Passwort setzen
- Role mapping & Groups: Rollen und Gruppen zuweisen
- Sessions zeigt alle aktiven Sessions des Benutzers an

Rollen

- In der Admin Console über "Realm roles/Create role" im richtigen Realm
- Realm Roles (für alle Clients) vs. Client Roles (nur für einen Client)
- Können ebenfalls beliebige Attribute haben
- Compound Roles enthalten andere Rollen, erzeugen über "Actions/Add associated roles"

Zuweisen von Rollen

- Am Benutzer über "Role mapping/Assign role"
- Als Default-Rolle für alle Benutzer in einem Realm über "Realm roles/default-roles-<realm>/Assign role"
- Als Default-Rolle für Benutzer, die sich über die Selbstregistrierung anmelden über "Realm settings/User registration" auf dem Tab "Default roles"

Übung 06: Benutzer mit Rollen

- In unserem Realm wollen wir über Rollen zwischen Trainern und Schulungsteilnehmern unterscheiden.
- Lege eine Rolle **trainer** und eine Rolle **student** für Trainer bzw. Schulungsteilnehmer an.
- Lege je einen Benutzer für den Trainer Klaus Klöbner und den Schulungsteilnehmer Hermann Wohlgemut an.
- Die Benutzer sollen ein Initialpasswort erhalten, das sie bei der ersten Anmeldung ändern müssen.

Gruppen

- Rollen können einer Gruppe über "Role mapping" zugewiesen werden

- Können ebenfalls beliebige Attribute haben
- Hierarchie von Gruppen über "Child groups" möglich

Typische Nutzung

- Rollen werden zu Gruppen gebündelt
- Benutzer werden zu Gruppen hinzugefügt
- Falls es nur sehr wenige Rollen gibt, kann ggf. auf die Gruppenebene verzichtet werden. Stattdessen können Benutzern dann direkt Rollen zugewiesen werden.

LDAP & AD

- Kerberos- und LDAP-Provider können über "Configure/User federation" hinzugefügt werden
- Grundsätzlich wird immer zuerst die lokale Keycloak-Datenbank abgefragt und dann der externe User-Provider
- Fällt ein Provider aus, ist auch eine Authentifizierung mit niedriger priorisierten Providern nicht mehr möglich
- Mapping im Default: username, email, first name und last name, weitere individuelle Mappings konfigurierbar

Synchronisation

- Benutzer aus LDAP können entweder periodisch oder bei Zugriff in die lokale Keycloak-Datenbank übernommen werden
- Passwörter werden nicht synchronisiert, sondern im LDAP-Server geprüft
- Für das Hashing beim Ändern von Passwörtern ist der LDAP-Server zuständig!
- Rückrichtung ebenfalls möglich: In Keycloak angelegte Benutzer können nach LDAP synchronisiert werden ("Sync Registrations")

Übung 07: Anbindung AD

- Konfiguriere für den Realm **training** die Anbindung an ein Active-Directory-Verzeichnis.
- Mit folgenden Einstellungen kannst du eine Verbindung zu einem Active Directory herstellen:
 - Connection URL: **ldap://ad1**
 - Bind DN: **CN=Administrator,CN=Users,DC=ad1,DC=example,DC=com**
 - Bind credentials: **Password**
- Deaktiviere die Funktion "Sync Registrations", so dass weiterhin auch lokale Benutzer in Keycloak angelegt werden können.
- Überprüfe deine Konfiguration über die Benutzersuche in der Admin Console und über den Login in die Account-Console.
- Tipp: **docker compose exec ad1 ldapsearch** zeigt alle LDAP-Objekte in **ad1** an

Mapping von LDAP-Attributen

- Username LDAP attribute: `sAMAccountName`
- Search scope: One Level oder Subtree
- Über den Tab "Mappers" können LDAP-Attribute auf Keycloak-Attribute abgebildet werden

LDAP Filter

- Beispiel für Benutzer der Gruppe Simpsons:

```
(&(objectClass=person)(sAMAccountName=*)  
(|(memberOf=CN=Simpsons,CN=Users,DC=ad1,DC=example,DC=com)))
```

- `&` verknüpft mit UND
- `|` verknüpft mit ODER

Gruppen & Rollen

- Um Gruppen und Rollen aus LDAP zu übernehmen muss ein `group-ldap-mapper` bzw. `role-ldap-mapper` ergänzt werden.
- Preserve Group Inheritance deaktivieren für flache Gruppenstruktur
- User Groups Retrieve Strategy: `LOAD_GROUPS_BY_MEMBER_ATTRIBUTE`
- Membership LDAP Attribute: `member`
- Membership Attribute Type: `DN`
- Mode: `READ_ONLY`

Externe Identityprovider

- Federation konfigurieren über "Configure/Identity providers"
- Social Login über GitHub, Facebook, Google oder Twitter
- Integration von IAM-Lösungen von Geschäftspartnern über OpenID Connect v1.0 oder SAML v2.0
- Einbindung cloud-basierter Identity Services

Beispiel: GitHub

- Configure/Identity providers/Add provider/GitHub
- In neuem Tab:
 - GitHub Profil → Settings → Developer Settings → OAuth Apps
 - Register a new application

- Authorization callback URL aus Feld "Redirect URI" von Keycloak übernehmen
- Client ID und Client Secret von GitHub nach Keycloak übernehmen

Register Application



Settings / Developer Settings








Register a new OAuth application

Application name *

Something users will recognize and trust.

Homepage URL *

The full URL to your application homepage.

Application description

This is displayed to all users of your application.

Authorization callback URL *

Your application's callback URL. Read our [OAuth documentation](#) for more information.

☐ **Enable Device Flow**

Allow this OAuth App to authorize users via the Device Flow.
Read the [Device Flow documentation](#) for more information.

Register application
Cancel

Client Id & Secret



Settings / Developer Settings








Application created successfully

[Settings](#) / [Developer settings](#) / [training](#)

General
Optional features
Advanced

training


KeycloakTraining owns this application.
Transfer ownership

You can list your application in the [GitHub Marketplace](#) so that other users can discover it.

List this application in the Marketplace

0 users
Revoke all user tokens

Client ID
3a2941e763999f9c012b

Client secrets
Generate a new client secret

You need a client secret to authenticate as the application to the API.

Application logo


Upload new logo

Provider hinzufügen

The screenshot shows the Keycloak Admin Console interface. On the left is a dark sidebar with a menu. The top of the sidebar has a hamburger menu icon and the 'KEYCLOAK' logo. Below the logo, there's a dropdown menu currently showing 'demo'. The menu items are: Manage, Clients, Client scopes, Realm roles, Users, Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers (highlighted), and User federation. The main content area has a breadcrumb 'Identity providers > Add provider' and a title 'Add Github provider'. The form contains several fields: 'Redirect URI' with the value 'https://keycloak.traefik.me/realms/demo/broker/github/endpoint', 'Client ID' with '3a2941e763999f9c012b', 'Client Secret' (masked with dots), 'Display order' (empty), 'Base URL' (empty), and 'API URL' (empty). At the bottom of the form are 'Add' and 'Cancel' buttons. The top right of the console shows a user profile icon and the text 'admin'.

Sign in with GitHub

The screenshot shows the Keycloak login page. The background is a dark grey with a low-poly geometric pattern. At the top center, it says 'DEMO' followed by a red heart icon. A white login card is centered on the page. The card has the title 'Sign in to your account'. It contains a 'Username or email' input field with a user icon on the right, a 'Password' input field, a 'Remember me' checkbox, and a 'Forgot Password?' link. A blue 'Sign In' button is below these fields. Underneath the button, it says 'Or sign in with'. Below this is a GitHub icon and the text 'GitHub'. At the very bottom of the card, it says 'New user?' followed by a 'Register' link.

Verknüpfung der Accounts

The screenshot shows the Keycloak Admin Console interface. On the left is a dark sidebar with navigation links: Manage, Clients, Client scopes, Realm roles, Users (selected), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, and User federation. The main content area is titled 'Users > User details' and shows the user 'keycloaktraining'. A toggle switch is set to 'Enabled'. Below this are tabs for Details, Attributes, Credentials, Role mapping, Groups, Consents, Identity provider links (active), and Sessions. Under 'Linked identity providers', a table lists one provider: GitHub, with a 'Social login' button and a 'Unlink account' link. Below this, the 'Available identity providers' section is empty, stating 'No available identity providers.'

Übung 08: Social Login

- Konfiguriere für den Realm **training** die Funktion "Sign in with GitHub"
- Du kannst entweder über deinen GitHub-Account eine eigene OAuth App anlegen oder folgende Daten verwenden:
 - Client Id: **3a2941e763999f9c012b**
 - Client Secret: **f75db5573ff342b738467757dce6ebd5be092f01**
- Überprüfe deine Konfiguration über die Account Console

Organisationen

- Neues Konzept in Keycloak 26
- Customer Identity and Access Management (CIAM) mit Fokus auf Business-to-Business (B2B) Use-Cases
- Zusätzliche Strukturierung der Benutzer innerhalb eines Realms

Kernfunktionen von Keycloak Organizations

- Verwaltung der Mitglieder
- Onboarding per Einladungslink oder Identity Brokering
- Identity-First-Login mit organisationsspezifischen Schritten
- Übertragung organisationsspezifischer Claims für autorisierte Zugriffe

Organisationen aktivieren und anlegen

- Aktivieren über Configure/Realm settings/Organizations
- Neuer Menüeintrag Manage/Organizations um Organisationen zu verwalten
- Browser Flow wird automatisch auf "Identity First" umgestellt
- Zuordnung von Domains zu Organisationen um Mitglieder anhand ihrer E-Mail-Adresse zu gruppieren
 - Beispiel:
 - Name: `trion development GmbH`
 - Alias: `trion`
 - Domain: `trion.de`

Mitglieder verwalten

- Realm-User können einer Organisation zugeordnet werden
- Mitglieder können per Invite-Mail eingeladen werden
- Managed vs. Unmanaged User
 - **Managed User** sind Mitglied der Organisation und existieren nur solange die Organisation existiert und aktiv ist
 - **Unmanaged User** können Mitglied der Organisation sein, existieren aber unabhängig von ihr

Identity Provider zuweisen

- Identity Provider werden weiterhin im Realm konfiguriert, können dann aber einer Organisation zugewiesen werden
- Benutzer, die über einen Identity Provider angelegt werden, der einer Organisation zugeordnet ist, werden automatisch Mitglied dieser Organisation

Roadmap Organisationen

- Die Planung des Features findet sich im [Epic Keycloak Organizations](#)
- Geplante Features umfassen u.a.:
 - Organization Roles & Groups
 - Self-Service für Organisationen
 - Angepasste Templates für Organisationen

Kapitel 5. Clients

Client anlegen

- Anlegen über Manage/Clients/Create client
- Client authentication
 - On: "confidential access type", Client Secret, für Backends
 - Off: "public access type", kein Client Secret, für Frontends (JS)
- Valid redirect URIs, * wird als Wildcard unterstützt
- Valid post logout redirect URIs, falls Logout am IdP genutzt wird
- Web origins für CORS, * um alle Origins zuzulassen
- Unter Credentials kann das Client Secret abgerufen werden

Übung 09: Public Client

- Unter <https://www.keycloak.org/app> stellt das Keycloak-Projekt eine Test-App für die Anmeldung über Keycloak zur Verfügung.
- Lege einen Client mit Client-Id **test-app** an, konfiguriere dabei insbesondere die "Valid redirect URIs" und die "Web origins".
- Melde dich mit einem Benutzer aus dem Realm bei der Test-App an, bei einer erfolgreichen Anmeldung zeigt sie den Namen des Benutzers

Konfiguration auf Client-Seite

- Discovery
 - `/realms/<realm>/well-known/openid-configuration`
 - Endpunkt-URLs
 - Link auf Keys/Zertifikate
 - Unterstützte Protokolle und Algorithmen
- Client Id und Secret
- ggfs: Scopes, Username Attribute
- Konkrete Konfiguration stark abhängig vom verwendeten Framework/Plattform

Logout

- Ohne Logout beim IdP wird der User beim nächsten Login-Versuch typischerweise direkt wieder angemeldet, um dies zu verhindern, sollte die Anwendung den Logout an den IdP propagieren.
- OIDC Spezifikationen:

- Session Management: Browser-basiert, regelmäßiges Polling auf Keycloak, Logout in der Anwendung sobald Session terminiert wurde
- RP-Initiated Logout: Browser-basiert, Logout wird von Anwendung initiiert
- Front-Channel Logout: iframe-basiert, propagiert Logout von Keycloak zur Anwendung
- Back-Channel Logout: Keycloak propagiert Logout per HTTP POST zur Anwendung

Client Registration Service

- Applikationen oder Services können selbst den Client in Keycloak anlegen
- Authentifizierung über Bearer-Token oder "Initial access token" (erzeugen unter Clients, Tab "Initial access token")
- Client Registration Policies definieren Rahmenbedingungen für authentifizierte und unauthentifizierte Requests gegen den Client Registration Service
- Details in [Securing Apps](#)

Kapitel 6. Mehrfaktorauthentifizierung

MFA

- Keycloak unterstützt als zweiten Faktor neben dem Passwort out of the box TOTP/HOTP über Mobile Apps und WebAuthn.
- Weitere Verfahren können über ein Service Provider Interface (SPI) entwickelt und eingebunden werden.

TOTP/HOTP

- Konfiguration über "Authentication/Policies/OTP Policy"
 - Time Based (TOTP)
 - Counter Based (HOTP)
- Einrichten über Admin Console (Users/Required user actions/Configure OTP) oder durch Benutzer in der Account Console
- Mobile Apps: [FreeOTP](#), [Google Authenticator](#), [Microsoft Authenticator](#)
- Zum Ausprobieren: <https://totp.danhersam.com/>

Übung 10: TOTP

- Aktiviere für einen Benutzer TOTP über die Required Action "Configure TOTP"
- Melde dich mit diesem Benutzer in der Account Console an und registriere einen TOTP Generator
- Melde dich ab und wieder an und prüfe, dass die Anmeldung nur akzeptiert wird, wenn ein gültiges One-Time-Password eingegeben wird.

WebAuthn

- W3C Standard
- Einrichten über Admin Console (Users/Required user actions/Webauthn Register bzw. Webauthn Register Passwordless)
- Unterstützt Hardware Tokens (z.B. [YubiKey 5](#)) und Passkeys
- Chrome Developer Tools unterstützen virtuelle WebAuthn Authenticator: More Options > More tools > WebAuthn.

Authentication Flows

- Werden über "Configure/Authentication/Flows" verwaltet
- Ein Flow kann über "Action/Duplicate" dupliziert werden (Dropdown oben rechts)
- Über "Action/Bind flow" kann der Flow aktiviert werden

Übung 11: WebAuthn

- Erstelle und aktiviere einen Authentication Flow, der WebAuthn Passwordless unterstützt
- Lege einen neuen Benutzer mit einer Required Action "WebAuthn Register Passwordless" an
- Nutze den Chrome Browser, aktiviere den virtuellen WebAuthn Authenticator und melde dich an der Account Console an
- Registriere deinen WebAuthn Authenticator und nutze ihn dann zur Authentifizierung

Individuelle Lösungen

- Eigene Implementierungen sind über das Authentication SPI möglich
 - erlaubt u.a. eigene Authenticator, Required Actions zu entwickeln und einzubinden
- Dokumentation im [Server Developer Guide](#)
- Beispiele u.a. in [keycloak/examples](#)

Kapitel 7. Themes

Organisation von Themes

- Themes werden unterhalb von `themes` in einem Verzeichnis mit dem Schema `<theme name>/<theme type>` abgelegt
 - `themes/demo/login` für das Login-Theme mit dem Namen `demo`
 - Wenn Docker-Image verwendet wird: `/opt/keycloak/themes/`
- Alternativ: Verteilung als JAR und Installation über Verzeichnis `providers`
- Standard-Themes, die mit Keycloak ausgeliefert werden, liegen in `/opt/keycloak/lib/lib/main/org.keycloak.keycloak-themes-<version>.jar` und bei [GitHub](#)
- Theme für Account Console v3 liegt in `/opt/keycloak/lib/lib/main/org.keycloak.keycloak-account-ui-<version>.jar`

Theme Types

- account: Account Management Console
- admin: Admin Console
- email: Von Keycloak versendete E-Mails
- login: Login Formulare inkl. Passwort vergessen, Registrierung, etc.
- welcome: Willkommensseite

Woraus besteht ein Theme?

- HTML ([Freemarker Templates](#))
- Bilder in `resources/img`
- Message Bundles in `messages`
- CSS in `resources/css`
- Javascript in `resources/js`
- Theme Properties in `theme.properties`

Entwicklung von Themes

- Während der Entwicklung helfen die Startoptionen
 - `--spi-theme-static-max-age=-1`
 - `--spi-theme-cache-themes=false`
 - `--spi-theme-cache-templates=false`
- Deaktiviert Caching und erlaubt Testen von Änderungen über einfachen Browser-Reload

theme.properties

- Properties-Datei in `<THEME TYPE>/theme.properties` innerhalb des the `themes` Verzeichnisses.
 - `parent`: Eltern-Theme, von dem dieses Theme abgeleitet wird
 - `import`: Importiert Ressourcen von einem anderen Theme
 - `styles`: Leerzeichenseparierte Liste der einzubindenden CSS-Dateien
Hinweis: auch CSS-Dateien aus dem Parent müssen explizit mit aufgeführt werden
 - `locales`: Kommaseparierte Liste der unterstützten Locales

Beispiel: Custom CSS für Login

- `themes/demo/login/theme.properties`

```
parent=keycloak.v2

styles=css/styles.css css/demo.css
```

- `themes/demo/login/resources/css/demo.css`

```
#kc-header {
  color: #ff0000 !important;
}

#kc-header-wrapper {
  font-size: 99px;
}
```

Übung 12: Theme

- Erstelle ein eigenes abgeleitetes Login-Theme bei dem du das Hintergrundbild der Login-Seite austauschst.
- Folgendes CSS-Snippet kann als Inspiration dienen:

```
.login-pf body {
  background: url("https://images.unsplash.com/photo-1693773024865-d2623880567d")
    no-repeat center center fixed;
  background-size: cover;
  height: 100%;
}
```


Tipps & Tools

- [UI Customization](#) Guides
- Für Anpassungen an den Freemarker Templates: [Template Author's Guide](#)
- Unterliegende Datenmodelle sind nicht explizit dokumentiert, oft hilft der Quellcode, z. B. [Free-MarkerLoginFormsProvider](#) für Login Themes
- [Keycloakify](#) generiert React Templates für Keycloak

Kapitel 8. Plugins

Erweiterung von Keycloak

- Bestehende Funktionalität kann geändert bzw. ersetzt oder erweitert werden
- Dokumentation im [Server Developer Guide](#)

Tipps zur Entwicklung

- Keycloak über docker compose im Development Mode nutzen
- Nützliche Umgebungsvariablen setzen:
 - `DEBUG: 'true'`
 - `DEBUG_PORT: '*:8787'`
 - `KC_LOG_LEVEL: INFO` oder `KC_LOG_LEVEL: DEBUG`
- Maven `target`-Verzeichnis nach `/opt/keycloak/providers` mounten
- Bei Änderungen: `mvn verify` und Keycloak neu starten, kein hot reloading

Service Provider Interfaces

- Die verfügbaren Service Provider Interfaces (SPI) können in der Admin Console abgerufen werden (Startseite, Tab "Provider Info")
- Es wird unterschieden zwischen
 - *Single-implementation provider types*: Es gibt genau eine Komponente, die das Interface implementiert, z.B. `HostnameProvider`
 - *Multiple implementation provider types*: Mehrere Komponenten implementieren das Interface, z.B. `Authenticator`

Provider Info

SPI	Providers
actionTokenHandler	verify-email execute-actions reset-credentials idp-verify-account-via-email update-email
admin-realm-restapi-extension	clear-realm-cache clear-user-cache ldap-server-capabilities testLDAPConnection ui-ext user-storage clear-keys-cache
authenticationSessions	infinispan
authenticator	auth-cookie reset-credentials-choose-user direct-grant-validate-password webauthn-authenticator auth-spnego

Authentication SPI

- Erlaubt es neben den im Standard enthaltenen Authentifizierungsverfahren (Passwort, Kerberos, OTP, WebAuthn, etc.) weitere eigene Verfahren zu ergänzen.
- Das Registrierungsformular für neue Benutzer kann angepasst oder ersetzt werden.
- Required Actions, das heißt Aktionen, die der Benutzer nach der Anmeldung einmalig ausführen muss, können ergänzt werden.

Action Token Handler SPI

- Action Tokens sind spezielle Json Web Tokens (JWT), die es einem Benutzer erlauben eine bestimmte Aktion auszuführen.
- Im Standard werden folgende Aktionen unterstützt:
 - Passwort zurücksetzen
 - E-Mail-Adresse bestätigen
 - "Required Action" ausführen, z.B. Nutzungsbedingungen akzeptieren
 - Verknüpfung eines Accounts mit einem Account bei externen IdP bestätigen

Event Listener SPI

- Implementieren von `EventListenerProvider` und `EventListenerProviderFactory`
- Erlaubt es auf *User-Events* (Login, Registrieren, ...) und *Admin-Events* (Client erzeugen/ändern, Realm konfigurieren, ...) zu reagieren

```
public interface EventListenerProvider extends Provider {

    void onEvent(Event event);

    void onEvent(AdminEvent event, boolean includeRepresentation);

}
```

SAML Role Mappings SPI

- Erlaubt es SAML-Rollen, die von einem externen IdP geliefert werden, auf interne Rollen in Keycloak zu mappen
- `org.keycloak.adapters.saml.RoleMappingsProvider` implementieren:

```
public interface RoleMappingsProvider {

    // ...

    Set<String> map(final String principalName, final Set<String> roles);

}
```

Vault SPI

- Erlaubt es Keycloak, Secrets (z.B. das SMTP- oder LDAP-Bind-Password) aus externen Vaults zu lesen, wo sie sicher gespeichert werden
- Im Standard wird ein Vault auf Basis von Textdateien (`file`), vorallem zur Nutzung mit Kubernetes Secrets, und auf Basis von Java Keystores (`keystore`) unterstützt
- Konfiguration ist im Guide [Using a vault](#) beschrieben

Erweiterungen des Servers

- Es können REST-Endpunkte ergänzt werden (über `RealmResourceProviderFactory` and `RealmResourceProvider`)
- Es können neue SPIs definiert werden (über `org.keycloak.provider.Spi` und `ProviderFactory` and `Provider`)
- Es können neue JPA-Entities definiert werden, um so das Datenmodell zu erweitern (über `JpaEntityProviderFactory` und `JpaEntityProvider`)

Kapitel 9. REST API für Administration

Admin CLI

- Authentifizieren über `kcadm.sh config credentials`

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  config credentials --server http://localhost:8080 \
  --realm master --user admin --password admin
```

- Benutzer abfragen über `kcadm.sh get users`

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  get users -r demo
```

- Benutzer anlegen und Passwort setzen

```
docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  create users -r demo -s username=testuser -s enabled=true

docker compose exec keycloak /opt/keycloak/bin/kcadm.sh \
  set-password -r demo --username testuser --new-password testpassword
```

REST API

- Direkte Nutzung der REST API ermöglicht die Integration in eigene Anwendungen oder Skripte
- Basis URL: `{base url}/admin/realms` z.B. <https://keycloak.example.com/admin/realms>
- Dokumentation der Ressourcen unter <https://www.keycloak.org/docs-api/latest/rest-api/>
- Da sowohl `kcadm.sh` als auch die Admin Console diese API nutzen, lässt sich darüber grundsätzlich jeder Use-Case realisieren, der über die Kommandozeile oder die Admin Console zur Verfügung steht

Export und Import

- Export über Kommandozeile:

```
docker compose exec keycloak /opt/keycloak/bin/kc.sh export --dir /tmp/export
```

- Beim Import darf die Keycloak-Instanz nicht laufen!
- Import entweder über `kc.sh import` analog zu Export oder beim Startup über die Kommando-

zeilenoption `--import-realm` aus dem Verzeichnis `/opt/keycloak/data/import`

- Gut geeignet zum Aufsetzen von Regressionstests mit definiertem Satz von Testdaten
- Details unter [Importing and Exporting Realms](#)
- Konfiguration als JSON/YAML verwalten: [adorsys/keycloak-config-cli](#)

Kapitel 10. Anpassungen für Produktivbetrieb

Tipps

- Nicht die In-Memory-Datenbank aus dem Default verwenden
 - Stattdessen typischerweise MariaDB oder PostgreSQL, in Zukunft auch CockroachDB
- Backup einrichten und regelmäßig Restore testen
- Testumgebung bereitstellen, um Upgrades vor Inbetriebnahme ausgiebig testen zu können
- Prüfen, ob High Availability (Cluster-Betrieb) erforderlich ist
- Falls mehr als geringfügige Last erwartet wird: Lasttests durchführen
- Betrieb hinter Reverse Proxy (z.B. TLS-Terminierung, zusätzliche Absicherung Admin-Console, WAF)

Metriken und Monitoring

- Keycloak stellt Metriken über Micrometer bereit
- Metriken können über die Umgebungsvariable `KC_METRICS_ENABLED=true` aktiviert werden und können dann per HTTP unter `/metrics` abgerufen werden
- [Dashboards](#) und Alerts über Prometheus und Grafana
- Überwachung der Log-Ausgaben (z.B. Loki oder Elasticsearch)

Feature Flags

- Steuerung über `KC_FEATURES` bzw. `KC_FEATURES_DISABLED`
- Preview Features:
 - `persistent-user-session`
 - `recovery-codes`
 - `update-email`: Update Email Action mit Verifikation
- Details unter [Enabling and disabling features](#)

Kubernetes

- [Keycloak Operator](#)
- Unterstützt Deployment und Konfiguration von
 - Datenbank
 - Hostname
 - TLS-Zertifikate und private Schlüssel
- [Keycloak Realm Import](#)